

5주차 예습과제

📖 딥러닝 파이토치 교과서 6 - 6.1.4 장

p244-312

https://colab.research.google.com/drive/1VnBGLyI9Tc6CvXv9FbspL_YfVtkIITpW

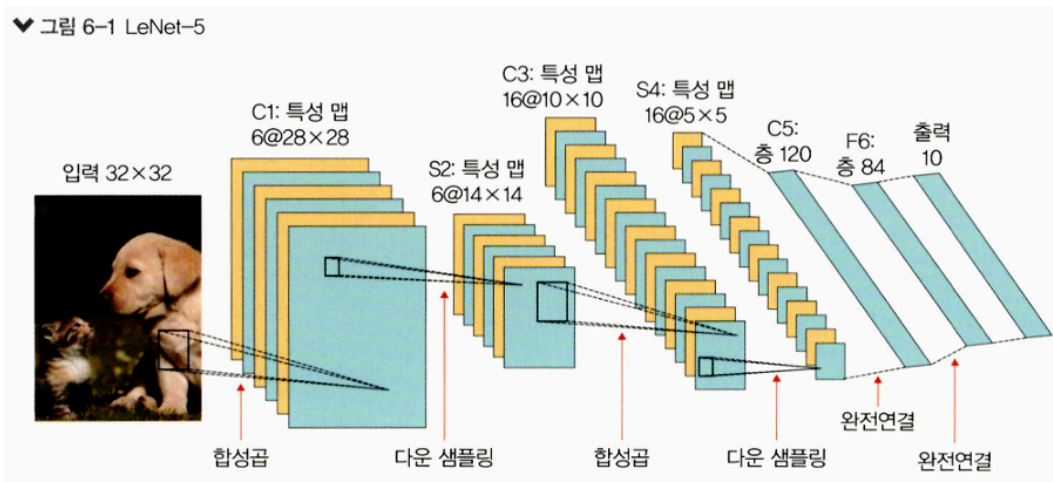
6장. 합성곱 신경망 II

6.1 이미지 분류를 위한 신경망

- 입력 데이터로 이미지를 사용한 분류(classification)는 특정 대상이 영상 내에 존재하는지 여부를 판단

6.1.1 LeNet-5

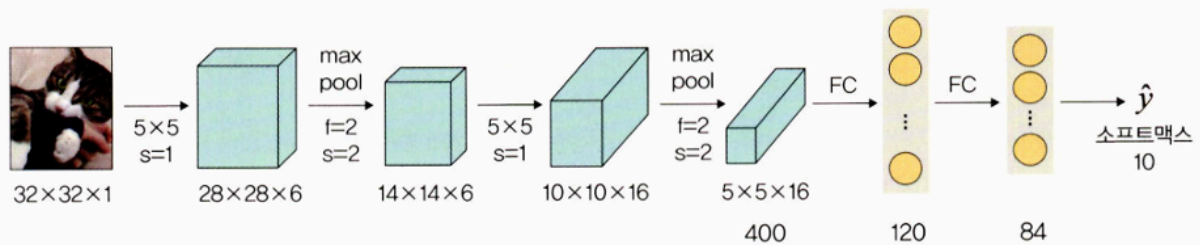
- LeNet-5
 - 합성곱 신경망이라는 개념을 최초로 얀 르쿤이 개발한 구조
 - 1995년 얀 르쿤, 레옹 보토, 요수아 벤지오, 패트릭 하프너가 수표에 쓴 손글씨 숫자를 인식하는 딥러닝 구조 LeNet-5를 발표
 - 현재 CNN의 초석이 됨
 - LeNet-5는 합성곱과 다운 샘플링(혹은 풀링)을 반복적으로 거치면서 마지막에 완전연결층에서 분류를 수행



- C1: 5×5 합성곱 연산 후 28x28 크기의 특성 맵 6개 생성
- S2: 다운샘플링하여 특성 맵 크기를 14x14로 줄임
- C3: 다시 5x5 합성곱 연산하여 10x10 크기의 특성 맵 16개를 생성
- S4: 다운 샘플링하여 특성 맵 크기를 5x5로 줄임
- C5: 5×5 합성곱 연산하여 1x1 크기의 특성 맵 120개 생성
- F6: 완전연결층으로 C5의 결과를 유닛(또는 노드) 84개에 연결
- C는 합성곱층, S는 풀링층, F는 완전연결층 의미

LeNet-5를 사용하는 예제 구현

▼ 그림 6-2 LeNet-5 예제 신경망



- 32×32 크기 이미지에 합성곱층과 최대 풀링층이 쌍으로 두 번 적용된 후 완전연결층을 거쳐 이미지가 분류되는 신경망

▼ 표 6-1 LeNet-5 예제 신경망 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	32×32	—	—	—
합성곱층	6	28×28	5×5	1	렐루(ReLU)
최대 풀링층	6	14×14	2×2	2	—
합성곱층	16	10×10	5×5	1	렐루(ReLU)
최대 풀링층	16	5×5	2×2	2	—
완전연결층	—	120	—	—	렐루(ReLU)
완전연결층	—	84	—	—	렐루(ReLU)
완전연결층	—	2	—	—	소프트맥스(softmax)

▼ 계층의 출력 크기 구하는 공식 (코드 6-10)

Conv2d 계층에서의 출력 크기 구하는 공식

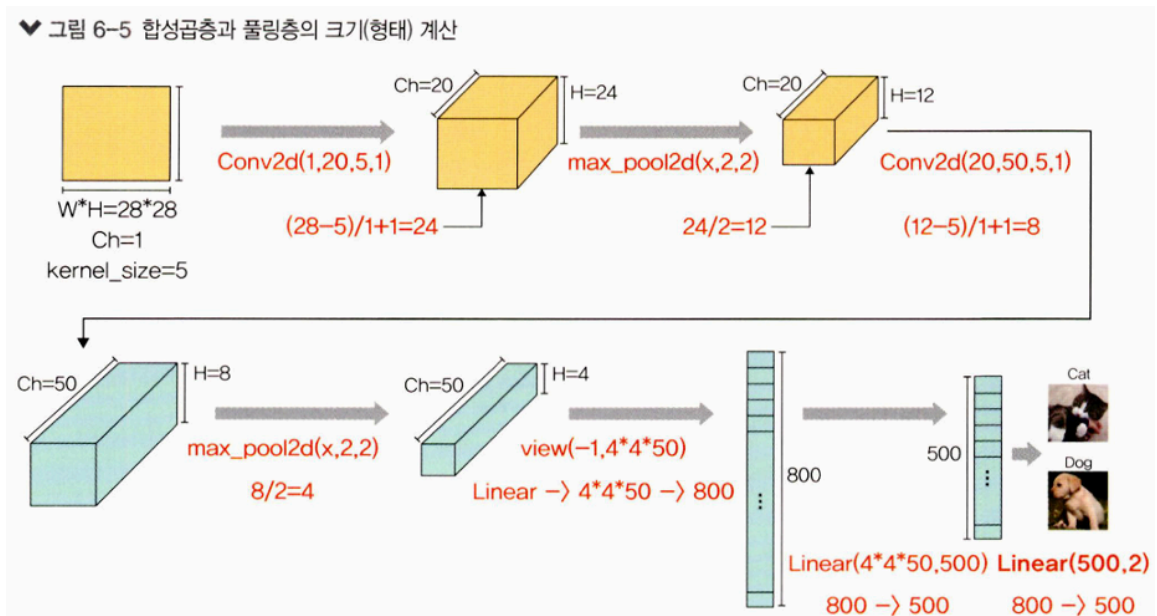
$$\text{출력 크기} = (W - F + 2P) / S + 1$$

- W : 입력 데이터의 크기(input_volume_size)
- F : 커널 크기(kernel_size)
- P : 패딩 크기(padding_size)
- S : 스트라이드(strides)

MaxPool2d 계층에서의 출력 크기 구하는 공식

$$\text{출력 크기} = IF / F$$

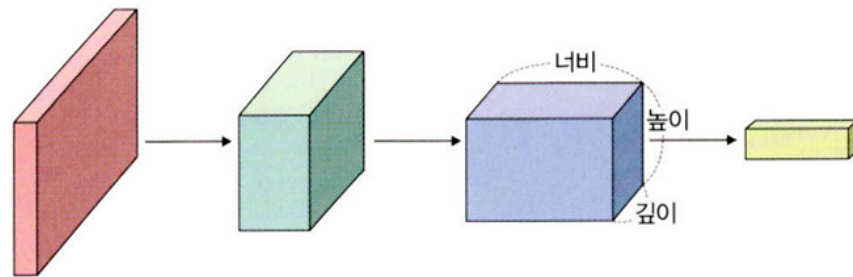
- IF : 입력 필터의 크기(input_filter_size, 또한 바로 앞의 Conv2d의 출력 크기이기도 합니다)
- F : 커널 크기(kernel_size)



6.1.2 AlexNet

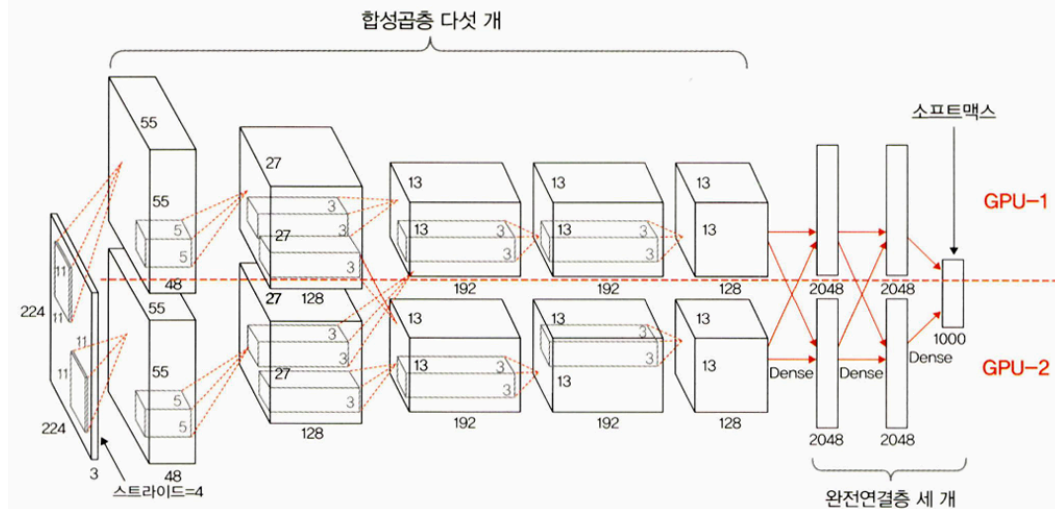
- ImageNet 영상 데이터베이스를 기반으로 한 화상 인식 대회 'ILSVRC 2012'에서 우승한 CNN 구조
- CNN은 너비, 높이, 깊이의 3차원 구조를 가지고, 합성곱을 거치면서 특성 맵이 만들어지고 이에 따라 중간 영상의 깊이도 달라짐

▼ 그림 6-8 CNN 구조



- AlexNet은 합성곱층 총 다섯 개와 완전연결층 세 개로 구성되어 있으며, 맨 마지막 완전 연결층은 카테고리 1000개를 분류하기 위해 소프트맥스 활성화 함수를 사용
- GPU 두 개를 기반으로 한 병렬 구조인 점 제외하면 LeNet-5와 비슷

▼ 그림 6-9 AlexNet 구조



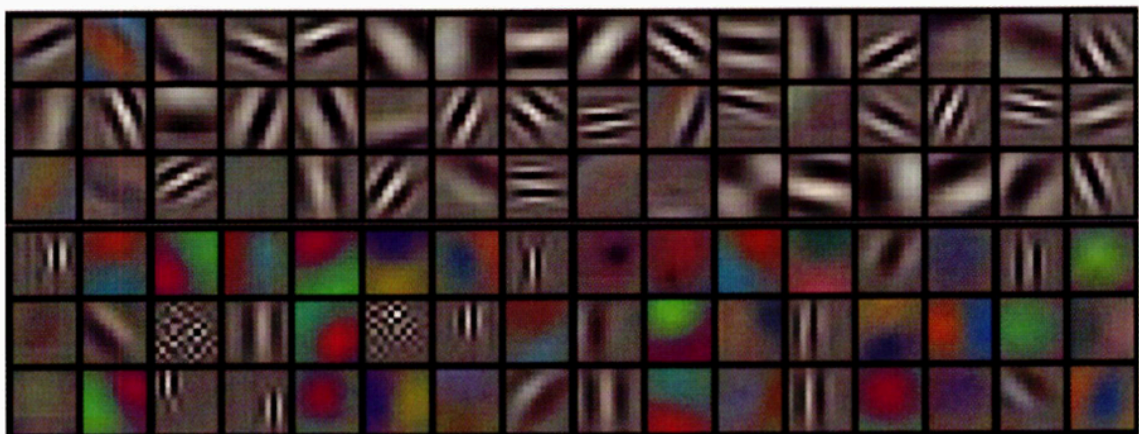
- 합성곱층에서는 렐루 활성화 함수를 사용

♥ 표 6-2 AlexNet 구조 상세

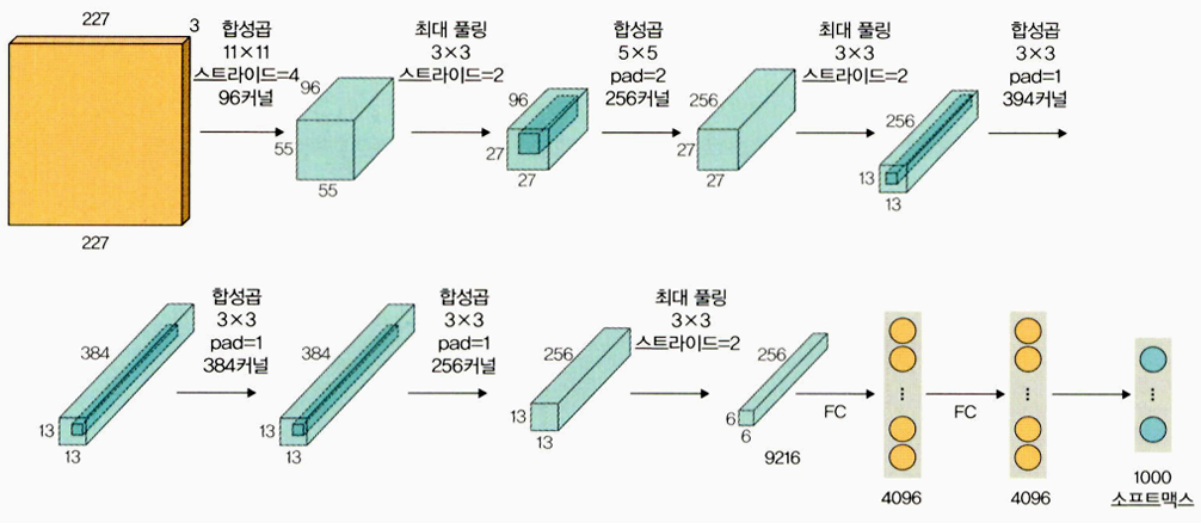
계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	227×227	—	—	—
합성곱층	96	55×55	11×11	4	렐루(ReLU)
최대 풀링층	96	27×27	3×3	2	—
합성곱층	256	27×27	5×5	1	렐루(ReLU)
최대 풀링층	256	13×13	3×3	2	—
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	256	13×13	3×3	1	렐루(ReLU)
최대 풀링층	256	6×6	3×3	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

- 네트워크에는 학습 가능한 변수가 총 6600만 개 있음
- 네트워크에 대한 입력은 227x227×3 크기의 RGB 이미지이며, 각 클래스에 해당하는 1000x1 확률 벡터를 출력
- 첫 번째 합성곱층 커널의 크기는 11x11×3이며, 스트라이드를 4로 적용하여 특성 맵을 96개 생성하기 때문에 55x55×96의 출력 가짐
- 첫 번째 계층을 거치면서 GPU-1에서는 주로 컬러와 상관없는 정보를 추출하기 위한 커널이 학습되고, GPU-2에서는 주로 컬러와 관련된 정보를 추출하기 위한 커널이 학습됨

♥ 그림 6-10 AlexNet GPU-1 · 2 적용 결과



▼ 그림 6-11 AlexNet 예제 네트워크



▼ `nn.AdaptiveAvgPool2d` (코드 6-29)

- `nn.AdaptiveAvgPool2d` 는 `nn.AvgPool2d` 처럼 풀링을 위해 사용
- `AvgPool2d` 에서는 풀링에 대한 커널 및 스트라이드 크기를 정의해야 동작함
 - `nn.AvgPool2d((3, 2), stride=(2, 1))` 처럼 지정하면 그 결과는 5x5 텐서를 3x3 텐서로, 7x7 텐서를 4x4 텐서로 줄이는 효과
 - (N, C, H_{in}, W_{in}) 크기의 입력을 (N, C, H_{out}, W_{out}) 크기로 출력하는 것이 `AvgPool2d`

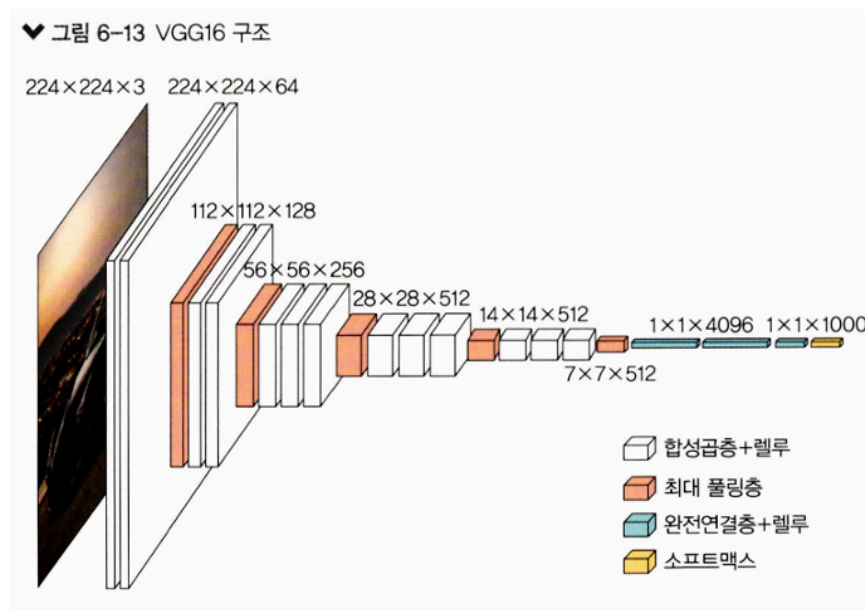
$$H_{out} = \left\lceil \frac{H_{in} + 2 \times \text{padding}[0] - \text{kernel_size}[0]}{\text{stride}[0]} + 1 \right\rceil$$

$$W_{out} = \left\lceil \frac{W_{in} + 2 \times \text{padding}[1] - \text{kernel_size}[1]}{\text{stride}[1]} + 1 \right\rceil$$

- `AdaptiveAvgPool2d` 는 풀링 작업이 끝날 때 필요한 출력 크기를 정의
 - `nn.AvgPool2d` 에서는 커널 크기, 스트라이드, 패딩을 지정했다면 `nn.AdaptiveAvgPool2d` 는 출력에 대한 크기만 지정
 - 앞의 수식에서 H_{out}, W_{out} 값이 정해졌기 때문에 커널 크기, 스트라이드, 패딩 값을 구할 수 있음
 - 따라서 `AdaptiveAvgPool2d` 를 사용할 경우 출력 크기에 대한 조정이 상당히 쉬워짐
 - `AdaptiveAvgPool2d` 는 입력 크기에 변동이 있고 CNN 위쪽에 완전연결층을 사용하는 경우 유용

6.1.3 VGGNet

- VGGNet은 카렌 시모니안과 앤드류 지서만이 2015 ICLR에 게재한 “*Very deep convolutional networks for large-scale image recognition*” 논문에서 처음 발표
- 합성곱층의 파라미터 수를 줄이고 훈련 시간을 개선하려고 탄생
- 즉, 네트워크를 깊게 만드는 것이 성능에 어떤 영향을 미치는지 확인하고자 나온 것
- 깊이의 영향만 최대한 확인하고자 합성곱층에서 사용하는 필터/커널의 크기를 가장 작은 3x3으로 고정함
- 네트워크 계층의 총 개수에 따라 여러 유형의 VGGNet(VGG16, VGG19 등)이 있음



- VGG16에는 파라미터가 총 1억 3300만 개 있음
- 모든 합성곱 커널의 크기는 3x3, 최대 풀링 커널의 크기는 2x2, 스트라이드는 2
- 결과적으로 64개의 224x224 특성 맵(224x224×64)들이 생성
- 마지막 16번째 계층을 제외하고 모두 ReLU 활성화 함수가 적용

▼ 표 6-3 VGG16 구조 상세

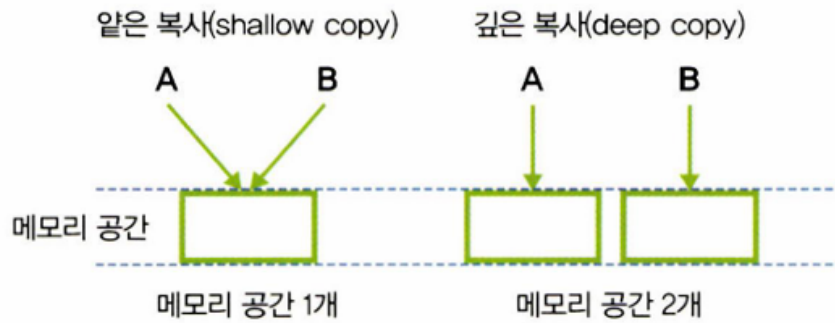
계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	224×224	—	—	—
합성곱층	64	224×224	3×3	1	렐루(ReLU)
합성곱층	64	224×224	3×3	1	렐루(ReLU)
최대 풀링층	64	112×112	2×2	2	—
합성곱층	128	112×112	3×3	1	렐루(ReLU)
합성곱층	128	112×112	3×3	1	렐루(ReLU)
최대 풀링층	128	56×56	2×2	2	—
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
최대 풀링층	256	28×28	2×2	2	—
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
최대 풀링층	512	14×14	2×2	2	—

○ 계속

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
최대 풀링층	512	7×7	2×2	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

▼ 얇은 복사와 깊은 복사 (코드 6-39)

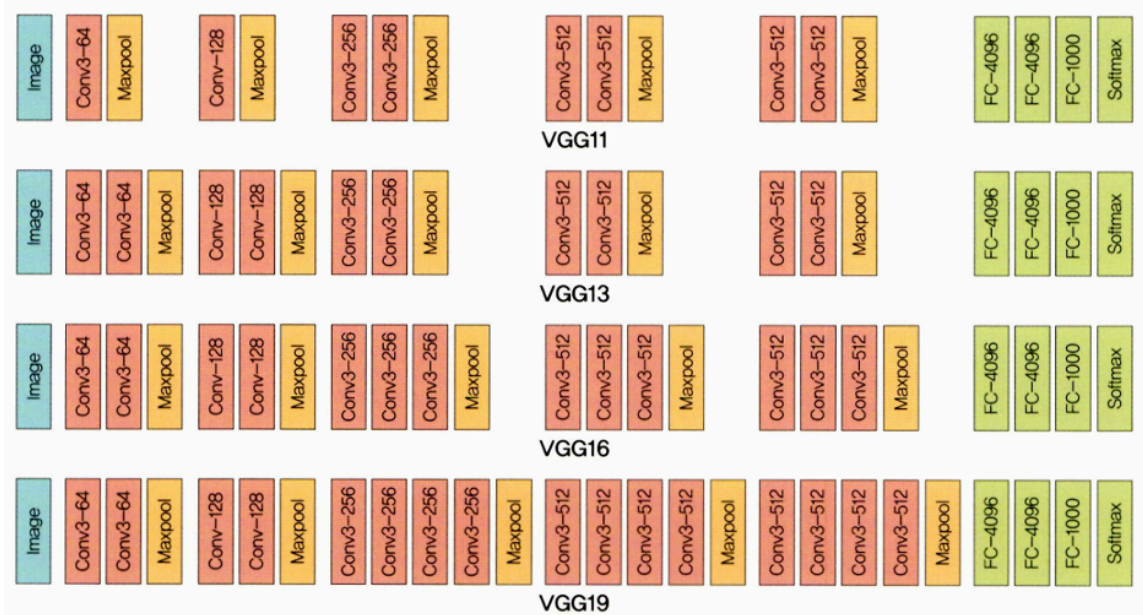
▼ 그림 6-14 얇은 복사와 깊은 복사



▼ VGG11, VGG13, VGG16, VGG19 모델의 계층 정리 (코드 6-41)

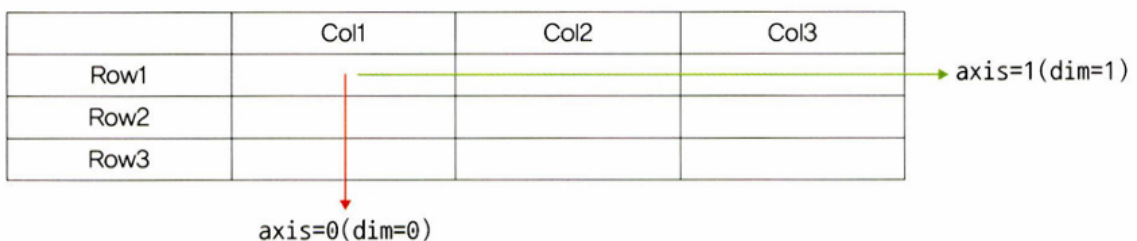
- 숫자(output channel, 출력 채널)는 Conv2d를 수행하라는 의미
- 출력 채널이 다음 계층의 입력 채널이 됨
- M은 최대 풀링을 수행하라는 의미

▼ 그림 6-15 VGG의 다양한 모델에 대한 네트워크



▼ 차원 (코드6-60)

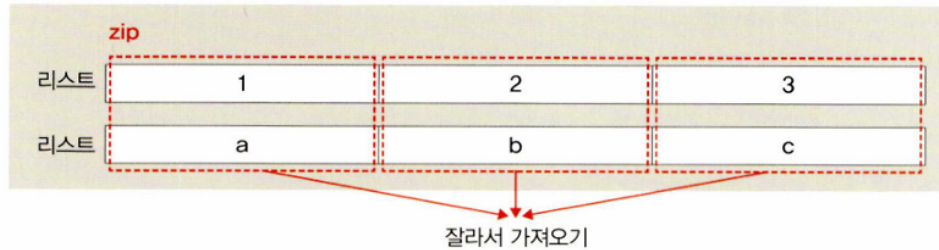
▼ 그림 6-17 차원(dim)



- `dim=0` 은 행, `dim=1` 은 열을 의미

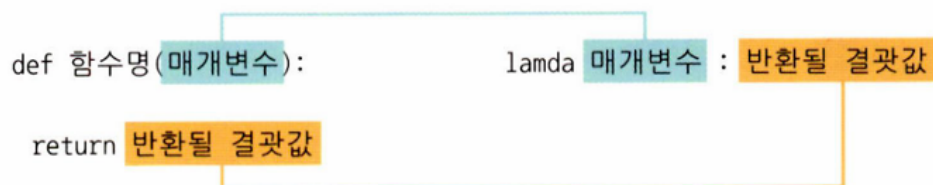
▼ zip() (코드 6-61)

▼ 그림 6-18 zip()의 원리



▼ 람다 함수 (코드 6-61)

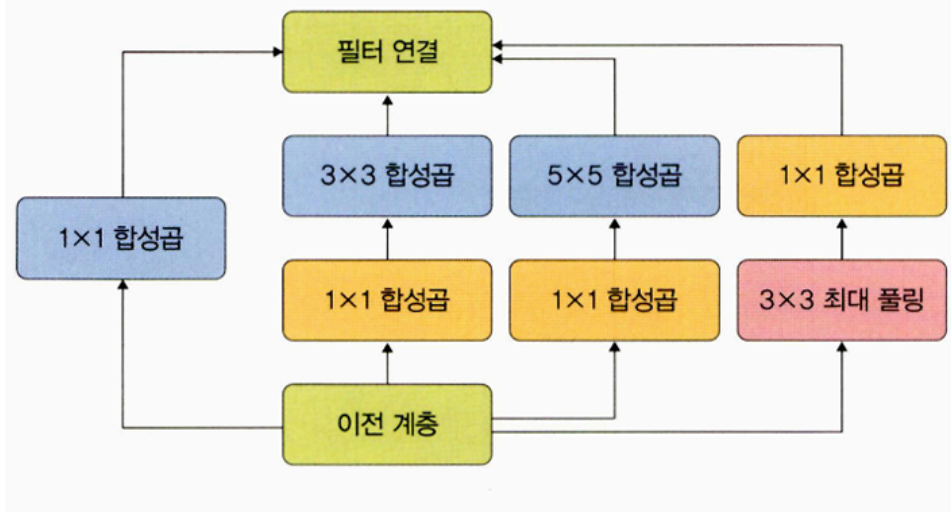
▼ 그림 6-19 일반 함수와 람다 함수



6.1.4 GoogLeNet

- GoogLeNet은 주어진 하드웨어 자원을 최대한 효율적으로 이용하면서 학습 능력은 극대화할 수 있는 깊고 넓은 신경망
- 깊고 넓은 신경망을 위해 GoogLeNet은 인셉션(inception) 모듈을 추가
 - 인셉션 모듈에서는 특징을 효율적으로 추출하기 위해 1x1, 3x3, 5x5의 합성곱 연산을 각각 수행
 - 3x3 최대 풀링은 입력과 출력의 높이와 너비가 같아야 하므로 풀링 연산에서는 드물게 패딩을 추가해야 함
- 결과적으로 GoogLeNet에 적용된 해결 방법은 희소 연결(sparse connectivity)
 - CNN은 합성곱, 풀링, 완전연결층들이 서로 밀집(정교하고 뽁뽁하게)하게 연결되어 있음
 - 뽁뽁하게 연결된 신경망 대신 관련성(correlation)이 높은 노드끼리만 연결하는 방법을 희소 연결이라고 함
 - 연산량이 적어지며 과적합도 해결 가능

♥ 그림 6-23 GoogLeNet의 인셉션 모듈



- 인셉션 모듈의 네 가지 연산
 - 1x1 합성곱
 - 1x1 합성곱 + 3x3 합성곱
 - 1x1 합성곱 + 5x5 합성곱
 - 3x3 최대 풀링 + 1x1 합성곱
- 딥러닝을 이용하여 ImageNet과 같은 대회에 참여하거나 서비스를 제공하려면 대용량 데이터를 학습해야 함
- 심층 신경망의 아키텍처에서 계층이 넓고(뉴런이 많고) 깊으면(계층이 많으면) 인식률은 좋아지지만, 과적합이나 기울기 소멸 문제를 비롯한 학습 시간 지연과 연산 속도 등의 문제가 있음
- 특히 합성곱 신경망에서 이러한 문제들이 자주 나타나는데, GoogLeNet(혹은 인셉션이라고도 불림)으로 이러한 문제 해결 가능